

Classes: An OO Design

Unit-08

Understanding **Object-Oriented Programming (OOP)** in C++17 involves diving into four fundamental concepts: **Encapsulation, Inheritance, Polymorphism, and Abstraction**

Encapsulation is about bundling the data (attributes) and methods (functions) that operate on the data into a single unit or class and restricting access to some of the object's components. This is achieved using access specifiers (**private**, **protected**, and **public**).

```
class Account {  
private:  
    double balance; // Private attribute  
  
public:  
    Account(double initialBalance) : balance(initialBalance) {} // Constructor  
  
    void deposit(double amount) { // Public method  
        if (amount > 0) balance += amount;  
    }  
  
    double getBalance() const { // Public method to access private attribute  
        return balance;  
    }  
};
```

Inheritance allows you to create a **new class (derived class)** that reuses, extends, and modifies the behavior of another **class (base class)**. This is a way to form a hierarchy of classes.

```
class SavingsAccount : public Account { // Inheritance
```

```
private:
```

```
    double interestRate;
```

```
public:
```

```
    SavingsAccount(double initialBalance, double rate)
        : Account(initialBalance), interestRate(rate) {}
```

```
    void addInterest() {
        double interest = getBalance() * interestRate / 100;
        deposit(interest);
    }
```

```
};
```

```
/*
SavingsAccount myaccount(1000, 7);
cout<< myaccount.get_balance() << endl; // 1000
myaccount.deposit(1000);
cout<< myaccount.get_balance() << endl; // 2000
myaccount.addInterest();// add interest = 140 to account
cout<< myaccount.get_balance() << endl; // 2140
*/
```

Inheritance allows you to create a **new class (derived class)** that reuses, extends, and modifies the behavior of another **class (base class)**. This is a way to form a hierarchy of classes.

When you specify `public` (or another access specifier like `private` or `protected`) while inheriting a class in C++, you are determining the access level of the base class members in the derived class. This is a key concept in C++ to control encapsulation and the inheritance hierarchy of classes. Let's explore what specifying `public` does:

Public Inheritance

- **Syntax:** `class Derived : public Base { ... };`
- In public inheritance, all public members of the base class remain public in the derived class, and all protected members of the base class remain protected in the derived class. Private members of the base class are not accessible directly from the derived class.
- Public inheritance is the most common form of inheritance in C++. It signifies an "is-a" relationship between the base class and the derived class. For instance, if `Dog` is a derived class of `Animal`, then public inheritance expresses that a `Dog` is a type of `Animal`.

Inheritance allows you to create a **new class (derived class)** that reuses, extends, and modifies the behavior of another **class (base class)**. This is a way to form a hierarchy of classes.

When you specify `public` (or another access specifier like `private` or `protected`) while inheriting a class in C++, you are determining the access level of the base class members in the derived class. This is a key concept in C++ to control encapsulation and the inheritance hierarchy of classes. Let's explore what specifying `public` does:

Public Inheritance

- **Syntax:** `class Derived : public Base { ... };`
- In public inheritance, all public members of the base class remain public in the derived class, and all protected members of the base class remain protected in the derived class. Private members of the base class are not accessible directly from the derived class.
- Public inheritance is the most common form of inheritance in C++. It signifies an "is-a" relationship between the base class and the derived class. For instance, if `Dog` is a derived class of `Animal`, then public inheritance expresses that a `Dog` is a type of `Animal`.

Inheritance allows you to create a **new class (derived class)** that reuses, extends, and modifies the behavior of another **class (base class)**. This is a way to form a hierarchy of classes.

Other Access Specifiers

- **Protected Inheritance (``protected``):** Public and protected members of the base class become protected in the derived class. This is less common and used in specific scenarios where you want to restrict access to the derived classes.
- **Private Inheritance (``private``):** Public and protected members of the base class become private in the derived class. This is used when you want to use the functionality of the base class but do not want to expose its interface. Private inheritance expresses a "has-a" or "is implemented in terms of" relationship rather than "is-a".

In summary, specifying ``public`` while inheriting a class in C++ is crucial for controlling access to the base class members, expressing the relationship between the base and derived classes, and ensuring that the principles of object-oriented programming, such as encapsulation and polymorphism, are correctly applied.

Inheritance method resolution.

In C++, if you want to call a function of a parent class instead of an overridden function in the current class, you can do so by explicitly specifying the parent class's name followed by the scope resolution operator (::) before the function name. This is useful in cases where you have a function in a base class that has been overridden in a derived class, but you still need to access the original version of the function from the derived class.

```
#include <iostream>

class Base {
public:
    void show() {
        std::cout << "Show Base class" << std::endl;
    }
};

class Derived : public Base {
public:
    void show() {
        std::cout << "Show Derived class" << std::endl;
        // Call the show function of the Base class
        Base::show();
    }
};

int main() {
    Derived d;
    d.show(); // This will call the show function of Derived class

    return 0;
}
```

Inheritance

method resolution.

```
int main() {  
    Derived d;  
    // Casting to base class reference and calling the base class function  
    static_cast<Base&>(d).show();  
  
    return 0;  
}
```

Using a Base Class Reference

```
int main() {  
    Derived d;  
    // Casting to base class pointer and calling the base class function  
    static_cast<Base*>(&d)->show();  
  
    return 0;  
}
```

Using a Base Class Pointer

Polymorphism allows methods to do different things based on the object it is acting upon. This is mainly achieved through virtual functions and function overriding in derived classes.

```
class Shape {
public:
    virtual double area() const = 0; // Pure virtual function
};

class Circle : public Shape {
private:
    double radius;

public:
    Circle(double r) : radius(r) {}

    double area() const override { // Overriding virtual function
        return 3.14159 * radius * radius;
    }
};
```

Abstractions is the concept of hiding the complex reality while exposing only the necessary parts. It's more of a design principle that affects how you structure your classes and interfaces (abstract classes in C++).

```
class Drawable {  
public:  
    virtual void draw() const = 0; // Pure virtual function providing an interface  
};
```

What is pure/non-pure virtual function?

```
class Base {
public:
    virtual void show() {
        std::cout << "Base show" << std::endl;
    }
};
```

show is a non-pure virtual function because it has its implementation. Derived classes can override show, but if they don't, calling show on a derived class object will execute the Base class version.

show is a pure virtual function (denoted by = 0), and Base becomes an abstract class. Any class that derives from Base must provide an implementation for show to be instantiable.

```
class Base {
public:
    virtual void show() = 0; // Pure virtual function
};
```

What is pure/non-pure virtual function?

Abstractions is the concept of hiding the complex reality while exposing only the necessary parts. It's more of a design principle that affects how you structure your classes and interfaces (abstract classes in C++).

```
// Abstract class (interface) for shapes
class Shape {
public:
    // Pure virtual function with a std::string argument for color
    virtual void draw(const std::string& color) const = 0;

    // Virtual destructor to ensure proper cleanup of derived objects
    virtual ~Shape() {}
};
```

An interface, in the context of object-oriented programming (OOP), is a concept that defines a contract within a system; it specifies a set of methods that a class must implement. Interfaces are about "what" actions can be performed, but they do not dictate "how" these actions are performed.

Abstractions is the concept of hiding the complex reality while exposing only the necessary parts. It's more of a design principle that affects how you structure your classes and interfaces (abstract classes in C++).

```
// Abstract class (interface) for shapes
class Shape {
public:
    // Pure virtual function with a std::string argument for color
    virtual void draw(const std::string& color) const = 0;

    // Virtual destructor to ensure proper cleanup of derived objects
    virtual ~Shape() {}
};
```

```
// Circle class that implements the Shape interface
class Circle : public Shape {
public:
    void draw(const std::string& color) const override {
        std::cout << "Drawing Circle in " << color << " color." << std::endl;
    }
};
```

Abstractions is the concept of hiding the complex reality while exposing only the necessary parts. It's more of a design principle that affects how you structure your classes and interfaces (abstract classes in C++).

```
void drawShape(const Shape& shape, const std::string& color) {  
    shape.draw(color);  
}  
  
int main() {  
    Circle circle;  
    Rectangle rectangle;  
  
    drawShape(circle, "Red");  
    drawShape(rectangle, "Blue");  
  
    return 0;  
}
```

Lets Test Our Skills

Exercise 1: The Animal Zoo

Objective: Practice inheritance and polymorphism.

1. Create a base class named `Animal` with a virtual function `makeSound()`.
2. Derive different animal classes from `Animal` (e.g., `Dog`, `Cat`, `Bird`) and implement their versions of `makeSound()` to print different messages.
3. Create a `Zoo` class that can hold a collection of `Animal`'s and has a method to make each animal in the zoo make its sound.

Exercise 2: Geometry Shapes Calculator

Objective: Use abstract classes and pure virtual functions.

1. Create an abstract base class `Shape` with pure virtual functions for calculating area and perimeter.
2. Create derived classes like `Circle`, `Rectangle`, and `Triangle`, implementing the area and perimeter calculations.
3. Write a program that can use a list of `Shape` pointers to calculate and display the area and perimeter of various shapes.

Exercise 3: Simple Bank Account Manager

Objective: Practice encapsulation and class relationships.

1. Design a `BankAccount` class with private attributes to store the account balance and public methods to deposit, withdraw, and display the current balance.
2. Extend this by creating derived classes such as `SavingsAccount` and `CheckingAccount`, where each class has specific behaviors, like minimum balance requirements or withdrawal limits.
3. Implement a `Bank` class that can manage multiple bank accounts, allowing you to add new accounts, delete accounts, and execute transactions.

Exercise 4: Book Library System

Objective: Work with associations and collections within classes.

1. Create a **Book** class with details like title, author, and ISBN number.
2. Develop a **Library** class that holds a collection of **Book** objects. Include methods to add a book, remove a book, and find books by author or title.
3. Optional: Implement a simple user interface in the console for interacting with the library system, adding books, and searching for them.

Exercise 5: Adventure Game Characters

Objective: Combine inheritance, polymorphism, and encapsulation in a fun context.

1. Design a base class `Character` with attributes like health, strength, and name. Include methods for actions like `attack` and `defend`.
2. Create derived classes for different character types (e.g., `Warrior`, `Mage`, `Archer`), each with unique abilities or stats.
3. Implement a simple battle system where characters can fight each other, taking turns to attack based on their specific strengths and weaknesses.

Exercise 5: Adventure Game Characters

Objective: Combine inheritance, polymorphism, and encapsulation in a fun context.

1. Design a base class `Character` with attributes like health, strength, and name. Include methods for actions like `attack` and `defend`.
2. Create derived classes for different character types (e.g., `Warrior`, `Mage`, `Archer`), each with unique abilities or stats.
3. Implement a simple battle system where characters can fight each other, taking turns to attack based on their specific strengths and weaknesses.

SELF STUDY

EXCEPTION HANDLING

SELF STUDY

EXCEPTION HANDLING

Exception handling in C++ is an essential feature that allows a program to deal with unexpected problems that might arise during its execution. By using exception handling, you can catch errors as they occur and handle them gracefully, potentially allowing the program to continue running instead of crashing.

In C++, exceptions are typically handled using a combination of three keywords: try, catch, and throw.

```
try {  
    // Code that might throw an exception  
}  
catch (const std::exception& e) {  
    // Code to handle the exception  
    std::cout << "Standard exception: " << e.what() << std::endl;  
}
```

Detailed Explanation:

1. `try` block:

- This block contains the code that might generate an exception. Any exception thrown within the `try` block is immediately transferred to the corresponding `catch` block.

2. `throw`:

- This keyword is used to throw an exception. You can throw any object as an exception, but it's standard practice to throw objects derived from the `std::exception` class, which provides useful methods like `what()` for error messages.

3. `catch` block:

- This block will handle the exception thrown by the `try` block. You can have multiple `catch` blocks to handle different types of exceptions. The `catch` block that matches the type of the exception thrown will be executed.